

Row Storage vs. Clustered Index: Un análisis experimental comparativo entre PostgreSQL y MySQL/InnoDB

Marlon E. García

Abstract: El presente informe analiza comparativamente la arquitectura de almacenamiento de PostgreSQL y MySQL/InnoDB, enfocándose en la diferencia entre Heap Storage (row storage desacoplado del índice primario) y Clustered Index (almacenamiento organizado por clave primaria). Se verifica su comportamiento bajo cargas OLTP (accesos puntuales y escrituras frecuentes) y OLAP (escaneos masivos y agregaciones), mediante observación y análisis de planes de ejecución y mediciones de rendimiento.

Los resultados demuestran que:

- PostgreSQL ofrece mayor flexibilidad en estrategias de acceso (Bitmap Heap Scan, Index Only Scan, HOT updates).
- InnoDB presenta ventajas en accesos por clave primaria debido al almacenamiento clusterizado.
- El costo de mantenimiento de MVCC difiere estructuralmente: PostgreSQL gestiona dead tuples, mientras InnoDB depende de Undo Logs y purge threads.
- En cargas OLTP con acceso por PK, InnoDB tiende a menor latencia.
- En cargas analíticas con filtros intermedios, PostgreSQL muestra planes más versátiles.

La elección óptima depende del patrón de acceso predominante, no únicamente del motor.

1. Introducción

Los sistemas gestores de bases de datos relacionales (RDBMS) modernos están diseñados para soportar cargas transaccionales (OLTP) y analíticas (OLAP) bajo las garantías ACID. No obstante, aunque comparten fundamentos teóricos comunes, se diferencian significativamente en su organización física del almacenamiento, lo que impacta directamente en el rendimiento ante distintos escenarios de acceso.

Dos enfoques predominan en motores comúnmente utilizados: el modelo *Heap Storage*, implementado por PostgreSQL, y el modelo *Clustered Index*, utilizado por el motor InnoDB en MySQL. Estas diferencias no son solo conceptuales, sino que afectan la localidad de datos, la fragmentación, el costo de mantenimiento y las decisiones del optimizador.

En el caso de InnoDB, la documentación oficial indica que el motor almacena los datos de la tabla dentro de un índice clusterizado basado en la clave primaria (PK), optimizando el acceso por dicha clave [1]. Este diseño implica que la tabla y el índice primario son físicamente la misma estructura B+Tree, lo que disminuye el número de accesos necesarios para búsquedas por clave primaria.

Por otro lado, PostgreSQL implementa un modelo de almacenamiento tipo heap, donde las tablas no están ordenadas físicamente por ningún índice. En este esquema, los índices contienen punteros hacia ubicaciones físicas (TID) dentro del heap. Este desacoplamiento entre

almacenamiento e indexación proporciona flexibilidad, pero puede requerir accesos adicionales al heap tras la navegación del índice.

Ambos motores implementan *Multi-Version Concurrency Control* (MVCC), aunque con mecanismos internos distintos. La documentación oficial de PostgreSQL describe MVCC como un sistema que permite que cada transacción observe una versión consistente de los datos en un momento determinado, independientemente de modificaciones concurrentes [2]. En PostgreSQL, las actualizaciones generan nuevas versiones físicas de las filas, dejando versiones anteriores que deben ser limpiadas mediante procesos de mantenimiento como `VACUUM`.

En contraste, InnoDB mantiene versiones anteriores utilizando estructuras de *Undo Log*, lo que evita la duplicación directa de filas en el archivo principal de datos, pero introduce costos asociados a la gestión de logs y procesos de purga [1]. Esta diferencia en la implementación de MVCC tiene implicaciones en el crecimiento físico de las tablas, el consumo de almacenamiento y la estabilidad del rendimiento bajo cargas intensivas de actualización.

El análisis del comportamiento interno de ambos motores requiere el uso de herramientas de inspección del plan de ejecución. PostgreSQL proporciona el comando `EXPLAIN` y su variante `EXPLAIN ANALYZE`, que permiten visualizar el plan estimado y los tiempos reales de ejecución de una consulta [3]. Asimismo, las vistas del sistema como `pg_stat_user_tables` ofrecen métricas relevantes sobre número de tuplas vivas y muertas, facilitando la evaluación del impacto de MVCC en el almacenamiento físico, como se menciona en el artículo No. 27.2.19 de la documentación oficial [4].

El objetivo de este informe es analizar experimentalmente cómo el modelo de almacenamiento—heap versus clustered index— influye en el comportamiento bajo distintos patrones de acceso. En particular, se evaluará el impacto en consultas de tipo *point lookup*, *range scan* y escaneos completos, así como el efecto de operaciones de actualización masiva en contextos OLTP y OLAP.

Más que determinar qué motor es superior, este estudio busca comprender cómo el diseño físico del almacenamiento condiciona las decisiones del optimizador y cómo dichas decisiones se reflejan en métricas observables de rendimiento. La evidencia empírica permitirá establecer recomendaciones fundamentadas sobre el uso adecuado de cada arquitectura según la naturaleza del workload.

2. Marco Teórico

2.1. Heap Storage en PostgreSQL

PostgreSQL implementa un modelo de almacenamiento basado en *heap files*, donde las tablas se almacenan como conjuntos de páginas sin un orden físico determinado. En este esquema, los índices son estructuras independientes que contienen punteros hacia las tuplas almacenadas en el heap.

Cada entrada de índice contiene un *Tuple Identifier* (TID), el cual referencia la ubicación física exacta de la fila dentro de un archivo de datos. Este desacoplamiento entre almacenamiento e indexación permite que una tabla tenga múltiples índices secundarios sin reorganizar físicamente los datos base.

Este diseño proporciona flexibilidad estructural, pero implica que un acceso mediante índice requiere dos pasos: (1) navegación del B-Tree y (2) acceso adicional al heap para recuperar la

fila completa. Este comportamiento es observable mediante el comando `EXPLAIN`, que permite inspeccionar los nodos del plan de ejecución [3].

Cuando las filas son actualizadas, PostgreSQL no sobrescribe la versión existente, sino que crea una nueva versión física. Este comportamiento forma parte de su implementación de MVCC, descrita oficialmente como un mecanismo que permite que cada transacción observe una vista consistente de los datos en un momento específico [2]. Las versiones obsoletas permanecen en el heap hasta que son eliminadas por procesos de mantenimiento como `VACUUM`.

Las estadísticas relacionadas con número de tuplas vivas y muertas pueden consultarse mediante la vista `pg_stat_user_tables`, lo que permite evaluar el impacto del versionado en el almacenamiento físico [4].

2.2. Clustered Index en InnoDB

El motor InnoDB utiliza un modelo de *clustered index*, donde la tabla misma se organiza como un B+Tree basado en la clave primaria. Según la documentación oficial, InnoDB almacena los datos de la tabla dentro del índice clusterizado, optimizando el acceso por clave primaria [1].

En este esquema, las hojas del B+Tree contienen directamente los registros completos de la tabla. Esto significa que una búsqueda por clave primaria requiere únicamente un recorrido del árbol, sin necesidad de un acceso adicional a una estructura separada.

Los índices secundarios, en cambio, no almacenan punteros físicos directos, sino que contienen la clave primaria correspondiente a cada registro. Por tanto, una búsqueda mediante índice secundario implica dos pasos: (1) navegación del índice secundario y (2) búsqueda posterior en el clustered index usando la clave primaria.

Este diseño mejora la localidad de datos en accesos por clave primaria y en escaneos por rango ordenados por dicha clave. Sin embargo, también implica que cambios frecuentes en la clave primaria pueden generar reorganizaciones costosas del árbol y *page splits*.

2.3. Comparación del Modelo MVCC

Ambos motores implementan Multi-Version Concurrency Control (MVCC), pero con enfoques distintos.

PostgreSQL genera nuevas versiones físicas de las filas ante cada actualización, permitiendo que transacciones concurrentes accedan a versiones previas sin bloqueos [2]. Las versiones antiguas (dead tuples) permanecen en el heap hasta que son eliminadas por procesos de limpieza. Este enfoque puede provocar crecimiento físico de la tabla y afectar el rendimiento si no se ejecuta mantenimiento periódico.

InnoDB, por su parte, mantiene versiones anteriores mediante estructuras de *Undo Log*, permitiendo reconstruir versiones históricas sin duplicar directamente la fila en el archivo principal de datos [1]. Este mecanismo reduce el crecimiento visible del data file, pero introduce costos asociados a la gestión de logs y procesos de purga.

Desde una perspectiva arquitectónica:

- PostgreSQL externaliza el costo del versionado en el heap.
- InnoDB internaliza el costo del versionado en estructuras de log.

Estas diferencias impactan el uso de almacenamiento, la fragmentación y la estabilidad del rendimiento bajo cargas intensivas de actualización.

2.4. Impacto del Layout de Almacenamiento en OLTP vs OLAP

El diseño físico del almacenamiento influye directamente en el comportamiento bajo distintos workloads.

En escenarios OLTP, caracterizados por accesos puntuales por clave primaria y alta frecuencia de inserciones y actualizaciones, el clustered index de InnoDB puede reducir la latencia debido a la menor cantidad de accesos necesarios para recuperar una fila específica [1]. La organización física ordenada por clave primaria mejora la localidad de referencia y reduce operaciones de I/O en búsquedas directas.

En PostgreSQL, los accesos por índice implican un *heap fetch*, lo que introduce un acceso adicional. No obstante, el motor puede emplear estrategias avanzadas como *Index Only Scan*, siempre que el *visibility map* permita evitar accesos al heap, lo cual es observable mediante `EXPLAIN ANALYZE` [3].

En escenarios OLAP, caracterizados por escaneos masivos, agregaciones y filtros de baja selectividad, la diferencia entre heap y clustered index no es necesariamente determinante. PostgreSQL puede optar por *Sequential Scan* o *Bitmap Heap Scan* cuando el costo estimado de recorrer la tabla completa es menor que el uso de índices [3]. Este comportamiento no es intrínsecamente negativo, sino resultado de decisiones basadas en estimaciones de cardinalidad y costo.

Además, el crecimiento de tuplas muertas en PostgreSQL puede afectar el rendimiento de escaneos completos si no se mantiene adecuadamente el sistema, lo que puede evaluarse mediante métricas en `pg_stat_user_tables` [4].

2.5. Implicaciones para el Optimizador

El optimizador de consultas toma decisiones basadas en estadísticas, costos estimados y estructura física de los datos. Herramientas como `EXPLAIN` permiten inspeccionar estas decisiones [3].

La presencia de nodos como:

- *Seq Scan*
- *Index Scan*
- *Bitmap Heap Scan*

refleja la interacción entre selectividad, distribución de datos y layout físico.

En consecuencia, comprender el modelo de almacenamiento no solo es relevante desde una perspectiva teórica, sino que resulta esencial para interpretar correctamente los planes de ejecución y justificar decisiones de diseño en sistemas productivos.

2.6. Comparación Estructural entre Heap Storage y Clustered Index

La Tabla I resume las diferencias estructurales más relevantes entre PostgreSQL (Heap Storage) e InnoDB (Clustered Index), considerando almacenamiento físico, comportamiento de índices y gestión de versiones.

TABLE I
COMPARACIÓN ENTRE HEAP STORAGE (POSTGRESQL) Y CLUSTERED INDEX (INNO DB)

Característica	PostgreSQL (Heap)	InnoDB (Clustered Index)
Organización física	Archivo heap sin orden físico garantizado	B+Tree ordenado por clave primaria
Acceso por PK	Índice + Heap Fetch	Un solo recorrido del B+Tree
Índices secundarios	Contienen TID (puntero físico)	Contienen la clave primaria
Actualizaciones (MVCC)	Nueva versión física en heap (dead tuples)	Versiones previas en Undo Log
Mantenimiento	Requiere VACUUM periódico	Purge automático de Undo Log
Fragmentación	Posible crecimiento por tuplas muertas	Page splits en inserciones desordenadas
Escaneos masivos	Seq Scan / Bitmap Heap Scan	Full table scan del clustered index
Localidad por PK	No garantizada	Alta localidad física

2.7. Row-Oriented Storage vs Column-Oriented Storage

Aunque tanto PostgreSQL como InnoDB son motores orientados a filas (row-store), resulta relevante contextualizar su comportamiento frente a modelos columnar descritos en la literatura académica.

Abadi et al. [5] analizan experimentalmente las diferencias entre sistemas orientados a filas y sistemas orientados a columnas, demostrando que los column-stores presentan ventajas significativas en cargas analíticas (OLAP) debido a:

- Lectura selectiva de columnas relevantes.
- Mayor compresión.
- Mejor utilización de caché.

Los autores concluyen que, aunque los row-stores pueden optimizar consultas analíticas mediante índices y técnicas auxiliares, su diseño físico no está orientado naturalmente a cargas de agregación masiva.

En este contexto, tanto PostgreSQL como InnoDB comparten la característica de almacenar filas completas contiguas, lo cual favorece cargas OLTP donde se requiere acceder a múltiples columnas de un mismo registro. Sin embargo, en escenarios OLAP intensivos, la ausencia de almacenamiento columnar nativo puede limitar el rendimiento comparativo frente a motores diseñados específicamente para análisis.

Esta perspectiva teórica permite interpretar los resultados experimentales del presente estudio dentro de un marco más amplio: no solo se comparan heap versus clustered index, sino que ambos se sitúan dentro del paradigma row-store descrito por Abadi et al. [5].

3. Diseño del Experimento

3.1. Entorno Experimental

Los experimentos se ejecutarán en PostgreSQL y MySQL 8.0 (InnoDB), utilizando el mismo hardware y configuración de memoria para garantizar comparabilidad. Las pruebas se ejecutan desde cero, sin datos previos, y con generación sintética determinística mediante funciones nativas.

Las métricas evaluadas incluyen:

- Tiempo de ejecución real (EXPLAIN ANALYZE)
- Plan de ejecución
- Número de páginas leídas (buffers)
- Tamaño físico en disco
- Fragmentación de índice
- Número de tuplas muertas (PostgreSQL)

3.2. Justificación de los Casos de Prueba

Los experimentos fueron seleccionados para aislar efectos específicos del layout físico:

Experimento 1: Evalúa la diferencia estructural entre heap y clustered index bajo acceso puntual por clave primaria (OLTP clásico).

Experimento 2: Analiza cómo la naturaleza de la clave primaria afecta directamente la estructura B+Tree clusterizada de InnoDB.

Experimento 3: Demuestra el impacto del modelo MVCC basado en versionado físico en PostgreSQL.

Experimento 4: Simula una carga OLAP comparando acceso completo vs acceso parcial a columnas.

Experimento 5: Evalúa si un covering index puede aproximar comportamiento columnar en un row-store.

El diseño permite aislar:

- Localidad física
- Fragmentación
- Costos de mantenimiento MVCC
- Diferencias entre acceso row completo y parcial

4. Resultados y análisis:

4.1. Experimento 1: Consulta puntual por clave primaria (OLTP)

4.1.1. Objetivo

Comparar el comportamiento de PostgreSQL (Heap Storage) y MySQL 8.4 (InnoDB Clustered Index) ante una búsqueda puntual por clave primaria en una tabla de 10 millones de registros.

4.1.2. Consulta Evaluada

```
SELECT * FROM practica_oltp WHERE id = 5000000;
```

Resultados:

4.1.3. PostgreSQL

```
Index Scan using practica_oltp_pkey on practica_oltp
(cost=0.43 .. 8.45 rows=1 width=37)
(actual time=1.417 .. 1.419 rows=1 loops=1)
```

Buffers: shared hit=4 read=3

Planning Time: 11.436 ms

Execution Time: 5.838 ms

Interpretación:

- Acceso mediante Index Scan (B-Tree).
- Doble acceso físico: índice + heap.
- 7 páginas accedidas (4 en caché, 3 desde disco).
- Tiempo de ejecución: 5.838 ms.

4.1.4. MySQL 8.4 (InnoDB)

```
-> Rows fetched before execution
(cost=0 .. 0 rows=1)
(actual time=0.00119 .. 0.00128 rows=1 loops=1)
```

Interpretación:

- Acceso directo sobre índice clusterizado (B+Tree).
- No existe heap separado.
- Tiempo de ejecución: 0.00128 ms.

4.1.5. Comparativa Cuantitativa

Motor	Tiempo ejecución	Arquitectura de acceso
PostgreSQL	5.838 ms	Índice + Heap (doble acceso)
MySQL InnoDB	0.00128 ms	Índice clusterizado (acceso único)

4.1.6. Comparativa Cualitativa

- PostgreSQL separa índice y datos, generando doble acceso.
- InnoDB integra índice y datos en la misma estructura.
- Para cargas OLTP basadas en PK, InnoDB reduce I/O lógico.
- PostgreSQL ofrece mayor flexibilidad estructural pero con leve sobre costo en accesos puntuales.

4.1.7. Conclusión

InnoDB presenta menor latencia en consultas por PK debido a su índice clusterizado, mientras que PostgreSQL requiere acceso adicional al heap. No obstante, PostgreSQL mantiene tiempos bajos incluso con 10 millones de registros, confirmando eficiencia en cargas OLTP.

4.2. Experimento 2: Impacto del tipo de clave primaria en InnoDB

4.2.1. Objetivo

Evaluar el impacto del tipo de clave primaria en el rendimiento y almacenamiento de InnoDB, comparando:

- PK numérica secuencial (BIGINT AUTO_INCREMENT)
- PK UUID en texto (CHAR(36))

Cada tabla contiene 5,000,000 registros.

Resultados:

4.2.2. Tiempo total de inserción

Tabla	Tiempo de inserción
pk_bigint	2 min 4 s
pk_uuid	2 min 49 s

La tabla con UUID fue aproximadamente 45 segundos más lenta (+36%).

4.2.3. Tamaño en disco

Tabla	Tamaño total (MB)
pk_bigint	311.89 MB
pk_uuid	445.00 MB

La tabla con UUID ocupa aproximadamente 133 MB adicionales (+42%).

4.2.4. Fragmentación y estructura interna

pk_bigint

- Rows: 4,855,235
- Avg_row_length: 67 bytes
- Data_length: 327,041,024 bytes
- Data_free: 6,291,456 bytes

pk_uuid

- Rows: 4,972,046
- Avg_row_length: 93 bytes
- Data_length: 466,616,320 bytes
- Data_free: 6,291,456 bytes

Ambas tablas presentan el mismo `Data_free`, por lo que la diferencia de tamaño no se debe a fragmentación externa significativa, sino al mayor tamaño de la clave primaria.

4.2.5. Análisis Técnico

En InnoDB, la clave primaria es el índice clusterizado. Esto implica que:

- Los datos se almacenan físicamente ordenados por la PK.
- El tamaño de la PK impacta directamente en cada registro.
- Todas las páginas hoja del B+Tree contienen la fila completa.

BIGINT AUTO_INCREMENT

- Inserciones secuenciales.
- Menor división de páginas (page splits).
- Mejor localidad de referencia.

UUID CHAR(36)

- Inserciones aleatorias.
- Mayor probabilidad de page splits.
- Árbol B+ más ancho y profundo.
- Mayor consumo de memoria y disco.

4.2.6. Conclusión

El tipo de clave primaria tiene impacto directo en:

- Tiempo de inserción (+36% con UUID)
- Tamaño en disco (+42%)
- Eficiencia del índice clusterizado

En cargas OLTP intensivas en escritura, una PK secuencial (BIGINT AUTO_INCREMENT) resulta significativamente más eficiente que UUID en formato texto, debido a la naturaleza clusterizada del almacenamiento InnoDB.

4.3. Experimento 3: VACUUM y Dead Tuples en PostgreSQL

4.3.1. Objetivo

Demostrar el efecto del modelo MVCC (Multi-Version Concurrency Control) de PostgreSQL mediante la generación de tuplas muertas (dead tuples) y su posterior limpieza con VACUUM.

4.3.2. Metodología

- Se creó una tabla con 10,000,000 registros.
- Se ejecutaron 1,000,000 actualizaciones.
- Se consultó `pg_stat_user_tables` para medir `n_dead_tup`.

- Se ejecutó `VACUUM`.
- Se volvió a medir el estado de las tuplas.

4.3.3. Resultados Antes de VACUUM

```
relname      : mvcc_test
n_live_tup   : 9,999,860
n_dead_tup   : 1,000,000
```

Tras 1 millón de `UPDATE`, PostgreSQL no sobrescribe las filas existentes. En su lugar:

- Crea una nueva versión de cada fila modificada.
- Marca la versión anterior como **dead tuple**.

Esto confirma el funcionamiento del modelo MVCC.

4.3.4. Ejecución de VACUUM

```
VACUUM mvcc_test;
Execute time: 1.788 s
```

4.3.5. Resultados Después de VACUUM

```
relname      : mvcc_test
n_live_tup   : 9,999,717
n_dead_tup   : 0
```

El contador de `n_dead_tup` se redujo a cero, lo que indica que:

- Las tuplas muertas fueron marcadas como espacio reutilizable.
- No se eliminaron físicamente del archivo (no hubo reducción automática de tamaño).

4.3.6. Análisis Técnico

En PostgreSQL:

- `UPDATE` = `INSERT` de nueva versión + invalidación de versión previa.
- Las versiones antiguas permanecen hasta que `VACUUM` las procesa.
- `VACUUM` no reduce el tamaño del archivo, solo libera espacio interno.

Esto explica por qué el tamaño en disco no disminuye tras un `VACUUM` estándar. Para reducir físicamente el tamaño sería necesario:

```
VACUUM FULL mvcc_test;
```

4.3.7. Conclusión

El experimento confirma:

- El modelo MVCC genera versiones múltiples de filas.
- Los `UPDATE` producen acumulación de dead tuples.
- `VACUUM` es esencial para mantener eficiencia y evitar bloat.
- El mantenimiento periódico es crítico en cargas OLTP intensivas.

Este comportamiento es estructural en PostgreSQL y constituye una diferencia clave frente a motores que implementan actualización en sitio.

4.4. Experimento 4: Full Scan y comportamiento columnar simulado

4.4.1. Objetivo

Evaluar el comportamiento de PostgreSQL ante consultas analíticas sobre una tabla de 10 millones de registros y 15 columnas, comparando:

- Query A: acceso a todas las columnas.
- Query B: acceso únicamente a dos columnas (agregación).

4.4.2. Consultas Evaluadas

```
-- Query A
SELECT * FROM ventas WHERE region = 'Norte';
```

```
-- Query B
SELECT SUM(total), COUNT(*)
FROM ventas WHERE region = 'Norte';
```

Resultados:

4.4.3. Query A:

Execution Time: 30809.300 ms
 Buffers: shared hit=14002 read=167817
 Parallel Seq Scan
 Rows returned: 2,500,013

Observaciones:

- Escaneo secuencial paralelo.
- Se leyeron 167,817 páginas desde disco.
- Acceso completo a las 15 columnas (width=460 bytes).
- Tiempo total: 30.8 s.

4.4.4. Query B:

Execution Time: 12335.411 ms
 Buffers: shared hit=14098 read=167721
 Parallel Seq Scan + Aggregate
 Rows returned: 1

Observaciones:

- También ejecuta Parallel Seq Scan.
- Se leyeron 167,721 páginas (prácticamente igual que Query A).
- Solo requiere 2 columnas (width=32 bytes).
- Tiempo total: 12.3 s.

4.4.5. Comparativa Cuantitativa

Métrica	Query A (SELECT *)	Query B (Agregación)
Tiempo ejecución	30.8 s	12.3 s
Páginas leídas (read)	167,817	167,721
Tipo de acceso	Parallel Seq Scan	Parallel Seq Scan
Columnas accedidas	15	2

4.4.6. Análisis Técnico

Ambas consultas realizaron un **escaneo secuencial completo** de la tabla.

Aunque Query B accede solo a 2 columnas:

- PostgreSQL utiliza almacenamiento por filas (row-store).
- Cada página contiene todas las columnas del registro.
- El motor debe leer la fila completa desde disco.

Por ello, el número de páginas leídas es prácticamente idéntico en ambas consultas. La reducción de tiempo (30.8 s → 12.3 s) se debe únicamente a:

- Menor volumen de datos procesados en memoria.
- No materialización de 2.5 millones de filas completas.

4.4.7. ¿Un Column Store sería más eficiente?

Sí. En un motor columnar:

- Solo se leerían las columnas `region` y `total`.
- Se evitaría la lectura de 13 columnas irrelevantes.
- El I/O físico se reduciría significativamente.

En este escenario, Query B tendría una reducción real en páginas leídas, no solo en procesamiento CPU.

4.4.8. Conclusión

El experimento demuestra que en PostgreSQL (row-store):

- El costo de I/O en consultas analíticas depende del tamaño total de la fila.
- Reducir columnas no reduce lectura física.
- Para cargas OLAP intensivas, un almacenamiento columnar resulta más eficiente.

Este resultado evidencia la diferencia estructural entre motores row-oriented y column-oriented en escenarios analíticos.

4.5. Experimento 5 (Bonus): Covering Index como aproximación columnar

4.5.1. Objetivo

Evaluar si un índice cubriente (covering index) puede simular el comportamiento de un motor columnar para una consulta analítica específica.

Se creó el índice:

```
CREATE INDEX idx_ventas_covering
ON ventas(region)
INCLUDE (total);
```

Se re-ejecutó:

```
SELECT SUM(total), COUNT(*)
FROM ventas
WHERE region = 'Norte';
```

4.5.2. Resultados Antes del Índice

```
Parallel Seq Scan
Buffers: shared hit=14098 read=167721
Execution Time: 12335.411 ms
```

Características:

- Escaneo secuencial completo.
- 167,721 páginas leídas desde disco.
- Tiempo total: 12.3 s.

4.5.3. Resultados Después del Covering Index

```
Parallel Index Only Scan using idx_ventas_covering
Heap Fetches: 12
Buffers: shared hit=18 read=12321 written=3458
Execution Time: 4304.893 ms
```

Características:

- Parallel Index Only Scan.
- Solo 12,321 páginas leídas (reducción 92%).
- Casi sin accesos al heap (12 heap fetches).

- Tiempo total: 4.3 s.

4.5.4. Comparativa Cuantitativa

Métrica	Sin índice	Con covering index
Tiempo ejecución	12.3 s	4.3 s
Páginas leídas (read)	167,721	12,321
Tipo de acceso	Seq Scan	Index Only Scan
Heap Fetches	N/A	12

4.5.5. Análisis Técnico

Con el índice cubriente:

- PostgreSQL evita leer la tabla completa.
- Solo accede al índice B-Tree.
- El índice contiene tanto `region` como `total`.
- Se elimina casi completamente el acceso al heap.

La reducción de páginas leídas (167k → 12k) demuestra una disminución real de I/O físico, no solo de procesamiento en memoria.

4.5.6. ¿Simula un Column Store?

Parcialmente sí, pero solo para esta consulta específica.

- El índice actúa como una estructura más angosta que contiene únicamente las columnas necesarias.
- Se evita la lectura de las otras 13 columnas.
- Sin embargo, sigue siendo un B-Tree orientado a filas, no almacenamiento columnar puro.

Un motor columnar almacenaría físicamente cada columna por separado, permitiendo esta optimización de forma natural y global, sin necesidad de índices adicionales.

4.5.7. Conclusión

El covering index redujo:

- El tiempo de ejecución en 65%.
- El I/O físico en 92%.

Esto demuestra que un índice cubriente puede aproximar el comportamiento de un motor columnar para consultas analíticas puntuales, aunque no reemplaza completamente un diseño column-oriented en cargas OLAP intensivas.

5. Conclusiones

El análisis comparativo entre PostgreSQL y MySQL/InnoDB permitió evidenciar que el rendimiento de un sistema gestor de bases de datos está profundamente condicionado por su arquitectura de almacenamiento y por las decisiones de diseño físico adoptadas a nivel interno.

Los modelos de heap storage e índice clusterizado representan enfoques estructuralmente distintos para organizar la información en disco. Mientras el almacenamiento tipo heap desacopla los datos de los índices, otorgando flexibilidad y menor dependencia estructural del diseño de la clave primaria, el índice clusterizado integra datos e índice en una única estructura B+Tree, optimizando accesos puntuales pero haciendo que la clave primaria determine el layout físico completo de la tabla.

Asimismo, el modelo de control de concurrencia basado en MVCC introduce implicaciones relevantes en el manejo de versiones, espacio en disco y mantenimiento. La generación de múltiples

versiones de filas favorece la concurrencia y reduce bloqueos, pero requiere estrategias activas de limpieza y administración para evitar degradación por acumulación de versiones obsoletas. Este comportamiento contrasta con esquemas de actualización en sitio donde el impacto se traslada principalmente a la estructura del índice y la fragmentación.

Desde la perspectiva analítica, los motores orientados a filas presentan limitaciones inherentes cuando las consultas acceden únicamente a subconjuntos reducidos de columnas, ya que el almacenamiento físico se organiza por fila completa. Aunque mecanismos como los índices cubrientes pueden mitigar parcialmente esta limitación, no sustituyen un modelo de almacenamiento columnar diseñado específicamente para cargas OLAP.

En conjunto, el estudio confirma que no existe un motor universalmente superior, sino arquitecturas optimizadas para distintos patrones de acceso. Las cargas transaccionales intensivas, con búsquedas frecuentes por clave primaria y alta tasa de inserciones secuenciales, se benefician de modelos clusterizados bien diseñados. Por su parte, entornos que requieren flexibilidad estructural, sofisticación del optimizador y manejo robusto de concurrencia encuentran ventajas en arquitecturas desacopladas basadas en heap y MVCC.

Comprender estas diferencias no es únicamente un ejercicio teórico, sino un requisito fundamental para el diseño eficiente de esquemas, la interpretación adecuada de planes de ejecución y la toma de decisiones informadas en entornos productivos. La arquitectura interna de un motor de base de datos no es un detalle de implementación, sino el factor determinante que condiciona su comportamiento observable bajo cargas reales.

Anexo A. Parte práctica

El script SQL utilizado para la parte Práctica Obligatoria de Row Storage vs Clustered Index se encuentra disponible en un repositorio público en línea [6].

References

- [1] Oracle Corporation, *InnoDB Storage Engine*, Oracle Corporation, 2024, *MySQL 8.0 Reference Manual*. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/innodb-index-types.html>
- [2] PostgreSQL Global Development Group, *Multiversion Concurrency Control (MVCC)*, PostgreSQL Global Development Group, 2024. [Online]. Available: <https://www.postgresql.org/docs/current/mvcc-intro.html>
- [3] —, *EXPLAIN*, PostgreSQL Global Development Group, 2024. [Online]. Available: <https://www.postgresql.org/docs/current/sql-explain.html>
- [4] —, *Monitoring Database Activity*, PostgreSQL Global Development Group, 2024. [Online]. Available: <https://www.postgresql.org/docs/current/monitoring-stats.html>
- [5] D. J. Abadi, S. R. Madden, and N. Hachem, “Column-stores vs. row-stores: How different are they really?” in *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*. ACM, 2008, pp. 967–980.
- [6] M. E. Garcia, “Row-storage-vs.-clustered-index,” GitHub repository, 2026, available online at: <https://github.com/MgarciaUMG/Row-Storage-vs.-Clustered-Index.git>.